

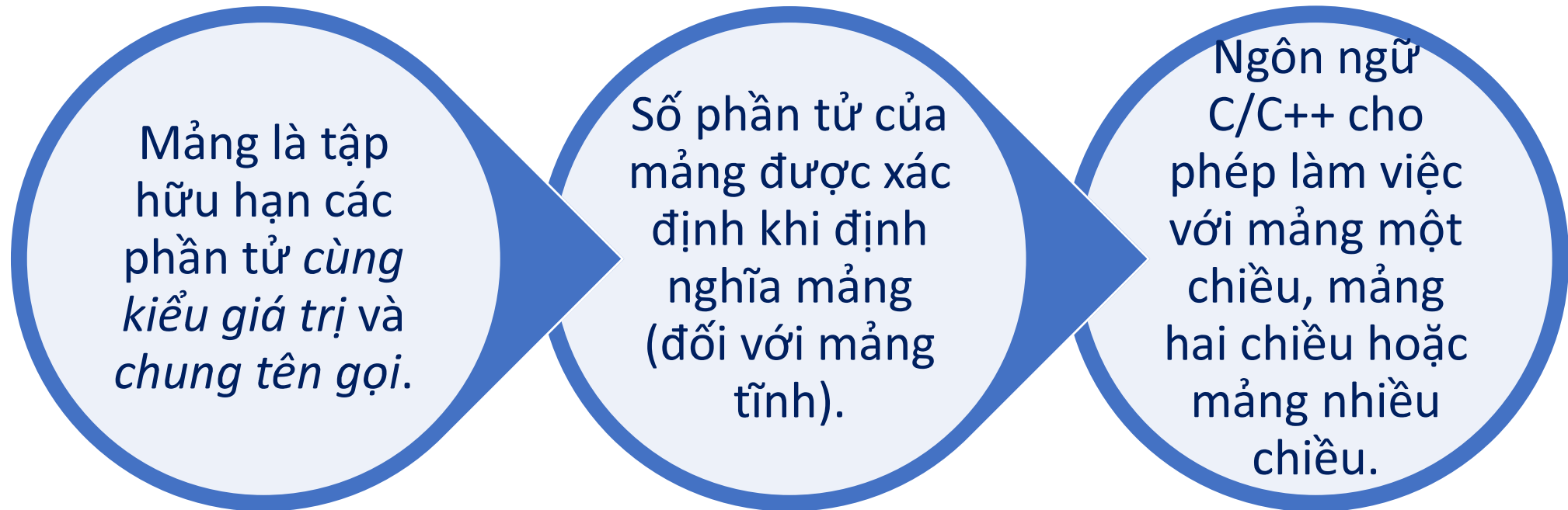
CHAPTER 4. ARRAYS

NỘI DUNG

- Giới thiệu cấu trúc dữ liệu mảng
- Mảng một chiều
- Mảng hai chiều
- Mảng vuông
- Mảng các ký tự
- Ví dụ và câu hỏi thảo luận

1. GIỚI THIỆU CẤU TRÚC DỮ LIỆU MẢNG

- Khái niệm



2.MẢNG MỘT CHIỀU

Khai báo mảng một chiều

< kiểu dữ liệu > tên mảng[const n];

Ví dụ để khai báo mảng một chiều có tên là a chứa 10 phần tử thuộc kiểu số nguyên ta thực hiện câu lệnh như sau:

```
int a[10];
```

Phần tử của mảng một chiều sẽ được truy xuất thông qua *tên mảng*[chỉ số], chẳng hạn để truy xuất phần tử thứ i của mảng một chiều có tên là a ta viết a[i].

Chỉ số của phần tử đầu tiên của mảng trong C/C++ mặc nhiên là 0, chẳng hạn trong khai báo trên mảng a gồm các phần tử a[0], a[1], ..., a[9].

Lưu ý: Trong một số tình huống, để dễ diễn đạt, chúng ta có thể sử dụng chỉ số đầu tiên của mảng là 1 hoặc là một giá trị khác mà chấp nhận lãng phí một số ô nhớ dành cho mảng.

Ví dụ 1.

Nhập các phần tử

```
void nhapmang(int a[], int &n)
{
    cin>>n;
    for (int i=0;i<n;i++)
        cin>>a[i];
}
```

Duyệt các phần tử

```
void duyetmang(int a[], int n)
{
    for (int i=0;i<n;i++)
        <Thực hiện thao tác trên phần tử  $a_i$ >
}
```

Sơ đồ tính tổng các phần tử của mảng (thiết kế theo kiểu có giá trị trả về)

```
int tinh tong(int a[], int n)
{
    int tong=0;
    for (int i=0;i<n;i++)
        if (a[i] thỏa mãn điều kiện bài toán)
            tong =tong+a[i];
    return tong;
}
```

(thiết kế theo kiểu không giá trị trả về)

```
void tinh tong(int a[], int n)
{
    int tong=0;
    for (int i=0;i<n;i++)
        if (a[i] thỏa mãn điều kiện bài toán)
            tong =tong+a[i];
    cout<<tong;
}
```

Sơ đồ đếm số phần tử thỏa điều kiện

```
int demso(int a[], int n)  
{  
    int dem=0;  
    for (int i=0;i<n;i++)  
        if (a[i] thỏa mãn điều kiện bài toán)  
            dem++;  
    return dem;  
}
```

Xóa phần tử tại vị trí k trong mảng

Để xóa một phần tử tại vị trí k của mảng, ta thực hiện như sau:

- Dịch chuyển các phần tử từ vị trí thứ $k + 1$ đến cuối mảng qua trái một vị trí.
- Sau khi dịch chuyển xong thì giảm số phần tử của mảng đi một đơn vị; nghĩa là cho $n = n - 1$.
- Do n thay đổi sau khi thực hiện hàm nên n cần được truyền theo kiểu tham biến.

```
void xoaphantu(int a[], int &n, int k)  
{  
    for (int i=k; i<n-1; i++)  
        a[i]=a[i+1];  
        n--;  
}
```

Chèn phần tử vào mảng

Chèn phần tử x vào tại vị trí thứ k của mảng ta thực hiện như sau:

- Dịch chuyển các phần tử từ vị trí thứ cuối mảng đến vị trí $k + 1$ qua phải một vị trí;
- tiếp theo chèn phần tử x vào vị trí k và cuối cùng là tăng n lên một đơn vị; nghĩa là $n = n + 1$.
- Cũng như hàm xóa ở trên; biến n phải được truyền theo kiểu tham biến.

```
void chenphantu(int a[], int &n, int k, int x)
{
    for (int i=n;i>k;i--)
        a[i]=a[i-1];
    a[k]=x;
    n++;
}
```

Đặt lính canh

- Phần tử lính canh có thể hiểu là một vị trí nào đó của mảng, hoặc là một giá trị nào đó của mảng hoặc cũng có thể là một giá trị tùy theo từng yêu cầu cụ thể của bài toán,...
- Tiến hành duyệt các phần tử của mảng, nếu có phần tử nào thỏa điều kiện của bài toán thì tiến hành thay đổi vị trí lính canh (hoặc giá trị canh).

Một số ví dụ về sơ đồ đặt lính canh

- Tìm giá trị lớn nhất
- Tìm giá trị nhỏ nhất
- Tìm giá trị nguyên tố lớn nhất (nếu không có số nguyên tố nào thì trả về giá trị ?)
- Tìm giá trị nguyên tố nhỏ nhất (nếu không có số nguyên tố nào thì trả về giá trị ?)

Sơ đồ đặt cờ hiệu

- Duyệt lần lượt từng phần tử của mảng, nếu có phần tử thỏa điều kiện thì dừng thuật toán và trả về một giá trị k nào đó;
- Nếu không có phần tử nào thỏa điều kiện thì trả về giá trị k' nào đó; giá trị k' mang ý đảo nghĩa với giá trị k ;
- Thường thì kỹ thuật cờ hiệu này nhằm trả lời cho câu hỏi: *có hay không ? đúng hay sai ? thuộc hay không thuộc ?....*
- Khác với kỹ thuật lính canh là luôn phải duyệt hết các phần tử của mảng, kỹ thuật cờ hiệu chỉ cần tìm một phần tử thỏa điều kiện.

```
int datcohieu(int a[], int n)  
  
{  
for (int i=0;i<n;i++)  
    if (a[i] thỏa mãn điều kiện bài toán)  
        return 0/1;  
  
return 1/0;  
  
}
```

Bài toán sắp xếp

Sắp xếp n phần tử của mảng 1 chiều theo thứ tự tăng dần

```
void sapxep(int a[], int n)
{
    for (int i=0; i<n-1;i++)
        for (int j=i+1;j<n ;j++)
            if (a[i]>a[j]) // sắp giảm thì thay dấu > bởi dấu <
                hoanvi(a[i],a[j])
}
```

trong đó hàm hoán vị được viết như sau:

```
void hoanvi(int &a, int &b)
{
    int temp=a;
        a=b;
        b=temp;
}
```

Bài toán tìm kiếm

Cho dãy n số nguyên $a_0, a_2, \dots, a_{n-1}$ và một số nguyên x . Hãy tìm xem x có thuộc vào dãy số trên hay không. Nếu tìm được ở vị trí thứ i thì xuất kết quả là i , ngược lại nếu không tìm thấy thì xuất kết quả là -1 (chú ý dãy bắt đầu từ chỉ số 0, nếu dãy có nhiều số bằng x thì xuất vị trí i nhỏ nhất).

```
int timkiem (int a[], int n, int x)
{
    int i = 0;
    while ( i<n  && a[i]!=x)
        i++;
    if( i==n)
        return -1; //    tìm hết nhưng không có x
    return i; //    tìm thấy a[i] là phần tử có khóa x
}
```

Q1. Sử dụng cấu trúc for viết lại thuật toán trên.

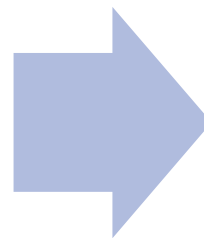
Q2. Hãy tìm các vị trí mà a[i] bằng x trong dãy.

3. MẢNG HAI CHIỀU

Khai báo mảng hai chiều

Cú pháp :

<kiểu dữ liệu> tên mảng[const m][const n];



Ví dụ:

```
int c[20][50];
```

Cấp phát bộ nhớ cho mảng

- Các phần tử trong mảng được cấp phát các ô nhớ kế tiếp nhau trong bộ nhớ.
- Biến mảng lưu trữ địa chỉ ô nhớ đầu tiên trong vùng nhớ được cấp phát.
- Kích thước của mảng bằng kích thước một phần tử nhân với số phần tử của mảng.
- Mảng hai chiều m dòng n cột được xem như là một bảng hình chữ nhật chứa $m*n$ phần tử cùng kiểu dữ liệu.
- ❖ Môi trường lập trình anh/chị đang sử dụng có thể khai báo được một mảng tối đa bao nhiêu phần tử ?

Hình ảnh một mảng hai chiều 3 dòng, 4 cột các số nguyên.

	cột 0	cột 1	cột 2	cột 3
dòng 0	2	3	4	6
dòng 1	3	8	4	7
dòng 2	3	1	2	5

Note: Ngôn ngữ C/C++ đánh chỉ số các phần tử của mảng từ vị trí 0 đối với mảng 1 chiều hoặc vị trí dòng 0 cột 0 đối với mảng 2 chiều

Truy xuất phần tử của mảng

- Các phần tử của mảng được truy xuất thông qua Tên mảng và Chỉ số/vị trí của phần tử trong mảng.

Khởi tạo giá trị cho mảng

Các phần tử của mảng có thể được khởi tạo giá trị ngay khi khai báo.

Ví dụ: `int a[4] = {1,4,6,2};`
`int b[2][3]={ {1,2,3}, {4,5,6} };`

Số lượng giá trị khởi tạo không được lớn hơn số lượng phần tử trong mảng.

Nếu số lượng này nhỏ hơn, các phần tử còn lại được khởi tạo giá trị 0.

```
int a[3][4] = { {1}, {4,5} };  
//a[0][0]=1,a[1][0]=4,  
a[1][1]=5 và các giá trị  
khác của mảng a đều  
bằng 0.
```

```
int a[3][4] = { }; //Tất cả  
đều mang giá trị 0
```

```
int b[] = {2, 4, 6, 8, 10, 12,  
14, 16}; //các phần tử từ  
b[8] trở đi được gán số  
nguyên ngẫu nhiên
```

Ví dụ 2.

Nhập mảng

```
1. void nhapmang(int a[maxm][maxn], int &m, int &n)
2. {
3.     cout<<"Nhap vao so dong: ";cin>>m;
4.     cout<<"Nhap vao so cot : ";cin>>n;
5.     for (int i=0;i<m;i++)
6.         for (int j=0;j<n;j++)
7.             cin>>a[i][j];
8. }
```

Xuất mảng

```
1. void xuatmang(int a[maxm][maxn], int m, int n)
2. {
3.     for (int i=0; i<m; i++)
4.     {
5.         for (int j=0; j<n; j++)
6.             cout<<a[i][j]<<" ";
7.         cout<<endl;
8.     }
9. }
```

Sơ đồ tính tổng

```
1. void tinhTong(int a[maxm][maxn], int m, int n)
2. {
3.     int tong=0;
4.     for (int i=0;i<m;i++)
5.         for (int j=0;j<n;j++)
6.             if (a[i] thỏa mãn điều kiện bài toán)
7.                 tong = tong +a[i][j];
8.     cout<<"Tong cac phan tu cua mang la : "<< tong;
9. }
```

Sơ đồ đếm phần tử thỏa điều kiện

```
1.int demso (int a[maxm][maxn], int m, int n)
2.{
3.    int dem=0;
4.    for (int i=0;i<m;i++)
5.        for (int j=0;j<n;j++)
6.            if (a[i] thỏa mãn điều kiện bài toán)
7.                dem++;
8.    return dem;
9.}
```

Ví dụ 2 (tiếp theo)

Cho mảng a gồm m dòng n cột, mỗi phần tử là một số nguyên.

1. Tìm tổng các phần tử của mảng.
2. Đếm số nguyên tố trong mảng.
3. Tìm giá trị nhỏ nhất của mảng.
4. Tìm giá trị lớn nhất của mảng.
5. Tính tổng các phần tử trên mỗi dòng.
6. Tính tổng các phần tử trên mỗi cột
7. Tìm giá trị lớn nhất trên mỗi dòng của mảng.
8. Tìm giá trị lớn nhất trên mỗi cột của mảng.

4. MẢNG VUÔNG

Ví dụ 3.

Cho mảng vuông có n dòng, n cột.

1. Tính tổng các phần tử nằm trên đường chéo chính
2. Tính tổng các phần tử nằm trên đường chéo phụ
3. Tính tổng các phần tử nằm phía trên đường chéo chính (tam giác trên)
4. Tính tổng các phần tử nằm phía dưới đường chéo chính (tam giác dưới)

5.MẢNG CÁC KÝ TỰ

KÝ TỰ

- Ký tự được đặt giữa hai dấu nháy đơn
- Mảng ký tự là mảng mà mỗi phần tử của nó chứa một ký tự (mảng ký tự gọi là một chuỗi)
- ❖ Ghi chú: Chi tiết về chủ đề chuỗi ký tự sẽ được nghiên cứu tiếp tục ở môn học kỹ thuật lập trình.

cctype.h

- void putchar (char c);
- void getchar(char c);
- int isalnum(int c);
- int isalpha(int c);
- int isdigit(int c);

- `int islower (int c);`
- `isupper(int c);`
- `int isspace(int c);`
- `int tolower(int c);`
- `int toupper(int c);`
- `int toascii(int c);`

NHẬP/XUẤT KÝ TỰ

- **Hàm `getchar()`**;//Dùng để đọc một ký tự từ bàn phím và trả về ký tự đó.
- **Hàm `getch()`**;//Có cùng chức năng như hàm `getchar()`, nhưng `getch()` nhập ký tự mà không hiện ra màn hình, trong khi `getchar()` nhập ký tự và ký tự đó có hiện ra màn hình.
- **Hàm `putchar()`**;// Xuất một ký tự ra màn hình.
- ❖ Có thể sử dụng các lệnh `cin`, `scanf` để nhập ký tự từ bàn phím; và các lệnh `cout`, `printf` để xuất ký tự lên màn hình.

NHẬP/XUẤT CHUỖI KÝ TỰ

- Nhập xuất/chuỗi ký tự: gets, puts
- Có thể sử dụng các lệnh cout, printf để xuất chuỗi ký tự.

❖ Lưu ý:

Sử dụng lệnh gets để nhập chuỗi; trước mỗi lệnh nhập chuỗi cần sử dụng lệnh xóa vùng đệm bàn phím bằng lệnh *flushall()* hoặc *fflush(stdin)*.

Ví dụ 5.

Cho mảng chứa các ký tự thỏa các điều kiện sau:

- Chỉ chứa các ký tự chữ cái thường và ký tự khoảng trắng.
- Đầu chuỗi và cuối chuỗi không chứa ký tự khoảng trắng.
- Giữa các từ có đúng một ký tự khoảng trắng.

Hãy viết chương trình thực hiện các công việc sau:

- Đếm xem trong mảng có bao nhiêu ký tự chữ hoa ? Bao nhiêu ký tự là chữ thường.
- Tìm mã ASCII của mỗi ký tự.
- Hãy chuyển các ký tự đầu từ thành chữ hoa còn các ký tự khác thành chữ thường.